

Primer Parcial: Criptografía, Firma Digital, Blockchain y VPN/HTTPS en Python

Yoann Esteban Jacome Garcia
Facultad de Ingeniería
Ingeniería en Multimedia
Universidad Militar Nueva Granada
Bogotá
1202597

David Santiago Crispoca Taborda
Facultad de Ingeniería
Ingeniería en Multimedia
Universidad Militar Nueva Granada
Bogotá
1202557

Maria Alejandra Suarez Cuesta
Facultad de Ingeniería
Ingeniería en Multimedia
Universidad Militar Nueva Granada
Bogotá
1202604

I. PUNTO 1: CRIPTOGRAFÍA CLÁSICA Y MODERNA

Cifrado César

1. Creación de las funciones de cifrado y descifrado: Desarrollamos una función capaz de cifrar cualquier texto usando un desplazamiento variable, es decir, la clave la elige el usuario en cada ejecución. La función recorre el mensaje y transforma cada letra según el valor de la clave, respetando mayúsculas, minúsculas y dejando intactos los espacios y signos. Un detalle que vale la pena mencionar es que no creamos una función separada para descifrar: aprovechamos que el cifrado César es simétrico, así que descifrar consiste simplemente en llamar a la misma función con el desplazamiento en negativo.

2. Prueba con nombre y frase: Probamos la función con un mensaje propio que incluye el nombre de uno de los integrantes, aplicando un desplazamiento de cinco. Verificamos que el texto cifrado es completamente ilegible y que, al descifrarlo con la misma clave, se recupera el mensaje original exactamente igual, incluyendo el formato.

3. Ataque de fuerza bruta: Implementamos el ataque aprovechando la principal debilidad del cifrado César: su espacio de claves es mínimo. Como el alfabeto tiene 26 letras, solo existen 26 desplazamientos posibles, así que programamos un ciclo que prueba todas las combinaciones y muestra los 26 resultados. Sin conocer la clave, basta con revisar la lista para identificar cuál línea contiene un mensaje con sentido. Esto demuestra que el cifrado puede romperse en segundos sin ninguna técnica sofisticada.

Cifrado Vigenère

Funciones de cifrado y descifrado con clave alfabética: Programamos una función que, en lugar de usar un número como clave, utiliza una palabra alfabética. Cada letra de la clave define un desplazamiento distinto, y la clave se repite en ciclo hasta cubrir todo el mensaje. La función está diseñada para cumplir doble propósito: mediante un parámetro de modo, la misma función cifra o descifra, invirtiendo la dirección del desplazamiento cuando es necesario.

1. Prueba con mensaje propio: Probamos la función con un mensaje propio y la clave “SEGURIDAD”, verificando que el texto cifrado no guarda relación visible con el original y que el descifrado con la misma clave lo recupera por completo.

2. Por qué dificulta el análisis de frecuencia: El análisis de frecuencia es una técnica que aprovecha que en cada idioma algunas letras aparecen más que otras. Con el cifrado César este análisis funciona porque cada letra siempre se reemplaza por la misma. Vigenère rompe ese patrón: como el desplazamiento cambia letra a letra según la clave, una misma letra del mensaje original puede aparecer cifrada de muchas formas distintas. Esto “aplana” las frecuencias y esconde las estadísticas del idioma, razón por la cual este cifrado fue considerado indescifrable durante siglos.

Cifrado simétrico moderno (AES)

1. Uso de la librería para cifrar y descifrar: Para el cifrado moderno usamos la librería de criptografía de Python en lugar de programar el algoritmo manualmente, que es como se trabaja en el mundo real. Generamos una llave aleatoria de 128 bits, ciframos un mensaje de prueba y luego lo desciframos exitosamente. A diferencia de los cifrados clásicos, aquí todo el proceso opera sobre datos binarios, por lo que el mismo procedimiento serviría igualmente para cifrar un archivo.

2. Por qué es más seguro y modos de operación ECB/CBC: La superioridad de AES frente a los cifrados clásicos se resume en tres aspectos: el tamaño del espacio de claves (2^{128} combinaciones frente a apenas 26 del César, lo que hace imposible la fuerza bruta), la resistencia al análisis estadístico, y la capacidad de cifrar cualquier tipo de dato digital. También implementamos los dos modos de operación principales. En el modo ECB cada bloque se cifra de forma independiente, lo que genera un problema grave: los bloques idénticos del mensaje producen bloques cifrados idénticos, dejando patrones visibles, por lo que no se recomienda su uso. En el modo CBC cada bloque se encadena con el anterior mediante un vector de inicialización aleatorio, de modo que incluso cifrar dos veces el mismo mensaje con la misma llave produce resultados completamente diferentes. Esto demuestra que en criptografía moderna no solo importa el algoritmo, sino también cómo se utiliza.

II. PUNTO 2: HASH Y HMAC

SHA-256

1. Aplicación de hashlib y cambios ante modificaciones del contenido: Usamos la librería hashlib para calcular el hash SHA-256 de distintos mensajes de texto. Para evidenciar el efecto avalancha, probamos tres mensajes: dos idénticos y uno con una sola letra modificada (una H mayúscula cambiada por minúscula). El resultado confirma que los mensajes idénticos generan exactamente el mismo hash, mientras que el mensaje con un solo carácter distinto produce un hash completamente diferente, sin ninguna similitud aparente con el original.

2. Importancia de la irreversibilidad y resistencia a colisiones: La irreversibilidad es fundamental porque un hash es una función de una sola vía: es computacionalmente inviable reconstruir el mensaje original a partir de su hash, lo que permite usarlo para almacenar contraseñas o huellas digitales de información sin exponer el contenido real. La resistencia a colisiones, por su parte, garantiza que sea extremadamente improbable que dos mensajes distintos produzcan el mismo hash; esto es lo que hace confiable usar un hash como “firma” única de un archivo o mensaje, ya que si dos personas pudieran generar el mismo hash con contenidos diferentes, se perdería la capacidad de verificar integridad y autenticidad.

Detección de integridad

1. Función para comparar hashes y verificar integridad: Creamos una función que calcula el hash SHA-256 de un archivo leyéndolo en bloques de 4096 bytes, lo que permite procesar archivos grandes sin cargarlos completos en memoria. Para probar la detección de integridad, generamos un archivo de prueba y calculamos su hash original. Luego simulamos una modificación añadiendo texto adicional al archivo y volvimos a calcular el hash. Al comparar ambos valores, la función identifica correctamente que el hash cambió, disparando la alerta de que el archivo fue modificado. Esto demuestra cómo el hash puede usarse como mecanismo de verificación de integridad: basta con comparar el hash actual de un archivo contra un hash de referencia guardado previamente para detectar cualquier alteración, por mínima que sea.

HMAC

1. Generación y verificación de HMAC con SHA-256 y clave: Programamos dos funciones: una para generar el código HMAC combinando un mensaje y una clave secreta mediante el módulo hmac con SHA-256 como algoritmo base, y otra para verificar si una firma recibida coincide con la que se generaría localmente usando el mismo mensaje y clave. Probamos el flujo completo generando una firma sobre un mensaje, confirmando que la verificación es exitosa cuando el mensaje no ha sido alterado, y comprobando que la verificación falla cuando se modifica incluso un solo carácter del mensaje original (agregando un signo de exclamación), a pesar de usar la misma clave.

2. Cuándo usar HMAC y diferencias con hash simple: Un hash simple (como SHA-256 aplicado directamente) solo garantiza integridad: permite detectar si un mensaje fue

modificado, pero no dice nada sobre quién lo generó, ya que cualquiera puede calcular el hash de un mensaje conocido y sustituirlo junto con su propio hash. HMAC resuelve esto incorporando una clave secreta compartida en el cálculo, de modo que solo quien conoce la clave puede generar o verificar una firma válida. Esto añade autenticidad además de integridad: HMAC se debe usar cuando se necesita confirmar que el mensaje proviene de una fuente legítima que posee la clave (por ejemplo, en comunicaciones entre servidores o APIs), mientras que un hash simple es suficiente cuando sólo se busca detectar corrupción o cambios accidentales en los datos, sin necesidad de autenticar el origen.

III. PUNTO 3: AUTENTICACIÓN CON HASH Y SAL

1. Simulación de registro/inicio de sesión guardando el hash: Implementamos un sistema que simula el registro y login de usuarios sin almacenar nunca la contraseña en texto plano. En el registro, se genera una sal aleatoria única para el usuario y se calcula el hash SHA-256 de la combinación sal + contraseña; en la base de datos (en este caso, en memoria) solo se guardan la sal y ese hash resultante. En el login, el sistema recupera la sal almacenada del usuario, recalcula el hash con la contraseña ingresada y compara ese resultado contra el hash guardado: si coinciden, el acceso es exitoso; si no, se rechaza. Probamos el flujo completo registrando un usuario e iniciando sesión tanto con la contraseña correcta (login exitoso) como con una incorrecta (login rechazado), incluyendo el caso de cambiar solo una letra de mayúscula a minúscula, lo que produce un hash completamente distinto y también rechaza el acceso.

2. Integración de la sal para evitar ataques de diccionario: Para demostrar la utilidad de la sal, registramos dos usuarios distintos con exactamente la misma contraseña y comparamos sus hashes almacenados: a pesar de compartir la contraseña, los hashes resultan completamente diferentes porque cada usuario tiene su propia sal aleatoria generada con `os.urandom`. Esto es clave porque, sin sal, dos usuarios con la misma contraseña tendrían el mismo hash, y un atacante podría precomputar los hashes de contraseñas comunes (ataque de diccionario) o usar tablas precalculadas (rainbow tables) para comparar contra toda la base de datos de una sola vez. Con sal, el atacante debe atacar cada hash individualmente, ya que cada uno depende de una sal distinta, lo que multiplica enormemente el costo del ataque.

3. Riesgos y la importancia de la sal: El riesgo principal que se evita es el uso de tablas precalculadas y ataques de diccionario a gran escala: si las contraseñas se hashean sin sal, un atacante que obtiene la base de datos puede comparar los hashes robados contra listas de hashes de contraseñas comunes ya calculadas de antemano, revelando muchas contraseñas casi instantáneamente. La sal neutraliza esta estrategia porque obliga a recalcular el hash específicamente para cada usuario, haciendo inútil cualquier tabla precomputada genérica. Además, es importante notar que la sal no necesita mantenerse en secreto (se puede guardar en texto plano junto al hash); su valor de seguridad no depende de ocultarla, sino de

que sea única y aleatoria por usuario. Por último, el sistema nunca “descifra” nada, ya que SHA-256 es una función de una sola vía: la verificación funciona únicamente repitiendo el cálculo y comparando resultados, lo que refuerza que ni siquiera un acceso directo a la base de datos expondría las contraseñas originales.

IV. PUNTO 4: PREGUNTAS REFLEXIVAS - INVESTIGACIÓN *Aleatoriedad y generadores de números aleatorios*

1. ¿Por qué es fundamental la aleatoriedad en criptografía? La aleatoriedad es la base de la seguridad criptográfica: se usa para generar claves, vectores de inicialización (IV), sales y nonces. Si un atacante puede predecir estos valores, puede anticipar o reproducir el proceso de cifrado y romper el sistema, sin importar qué tan robusto sea el algoritmo. La imprevisibilidad es lo que hace inviable adivinar o reconstruir claves mediante patrones.

2. Diferencia entre RNG y PRNG: Un RNG (generador de números verdaderamente aleatorios) obtiene aleatoriedad de fuentes físicas impredecibles (ruido térmico, movimientos del mouse, tiempos de interrupción del hardware). Un PRNG (pseudoraleatorio) usa un algoritmo determinista que, a partir de una semilla, genera una secuencia que parece aleatoria pero es reproducible si se conoce la semilla. En Python, `random` es un PRNG no seguro para criptografía (predecible), mientras que `os.urandom()` y el módulo `secrets` usan fuentes de entropía del sistema operativo, adecuadas para uso criptográfico.

Modelos de ataque y criptoanálisis clásico

1. Modelos COA, KPA, CPA, CCA: COA (Ciphertext-only): el atacante solo tiene acceso al texto cifrado y debe deducir la clave o el mensaje original solo a partir de eso. KPA (Known Plaintext): el atacante conoce pares de texto plano y su cifrado correspondiente, y busca deducir la clave para descifrar otros mensajes. CPA (Chosen Plaintext): el atacante puede elegir textos planos arbitrarios y obtener su cifrado, lo que le da más información para deducir la clave. CCA (Chosen Ciphertext): el atacante puede elegir textos cifrados y obtener su descifrado, siendo el modelo más poderoso y usado para evaluar la robustez de sistemas modernos.

2. Análisis de frecuencia en César y Vigenère: En el cifrado César, como cada letra siempre se sustituye por la misma, basta con comparar la frecuencia de letras del texto cifrado contra la frecuencia típica del idioma (por ejemplo, la “E” es la más común en español) para deducir el desplazamiento. En Vigenère, el análisis es más complejo porque cada letra puede cifrarse de forma distinta según su posición en la clave; técnicas como el método Kasiski o el índice de coincidencia se usan primero para estimar la longitud de la clave, y luego se aplica análisis de frecuencia por separado a cada subconjunto de letras cifradas con el mismo carácter de la clave.

3. Principio de Kerckhoff: Establece que la seguridad de un sistema criptográfico debe residir únicamente en el secreto de la clave, no en el secreto del algoritmo. Es vital porque permite que los algoritmos sean públicos, revisados y

auditados por la comunidad (lo que fortalece su confiabilidad), evitando depender de “seguridad por oscuridad”, que falla en cuanto el diseño se filtra o se reconstruye por ingeniería inversa.

Seguridad computacional y tamaño de llaves

1. ¿Qué implica “seguridad computacional”? Un sistema es computacionalmente seguro cuando romperlo (por ejemplo, mediante fuerza bruta) requiere una cantidad de recursos y tiempo tan grande que resulta impráctico, aunque teóricamente no sea imposible. Está directamente relacionado con el tamaño de la clave: cada bit adicional duplica el espacio de búsqueda. Por ejemplo, una clave de 128 bits (AES) tiene 2^{128} combinaciones posibles, un número tan grande que ni con las supercomputadoras actuales sería viable probarlas todas, mientras que una clave César de solo 26 posibilidades se rompe en milisegundos.

2. ¿Por qué no usar ECB con imágenes o archivos públicos? Porque ECB cifra cada bloque de forma independiente y bloques idénticos de texto plano producen bloques cifrados idénticos. En una imagen, grandes áreas de color uniforme (fondos, formas simples) generan bloques repetidos que, aunque estén “cifrados”, conservan el patrón visual original. El ejemplo clásico es la imagen del pingüino de Linux (Tux) cifrada con ECB: la silueta del pingüino sigue siendo perfectamente reconocible a pesar del cifrado, demostrando que ECB filtra la estructura del contenido.

Diferencias y riesgos: Criptografía, Esteganografía y Ofuscación

1. Comparación con ejemplos: Criptografía: transforma el mensaje en algo ilegible mediante un algoritmo y una clave (ej. cifrar un texto con AES); el mensaje cifrado es visible pero incomprensible sin la clave. Esteganografía: oculta la existencia del mensaje dentro de otro archivo (ej. esconder texto dentro de los píxeles de una imagen); un observador ni siquiera nota que hay un mensaje oculto. Ofuscación: hace que el código o los datos sean difíciles de entender o leer, pero no los protege matemáticamente (ej. minificar o renombrar variables en código fuente para dificultar su lectura).

2. ¿Por qué la seguridad por ocultamiento es una vulnerabilidad? Porque su protección depende únicamente de que el atacante no descubra el método usado (dónde está oculto el mensaje o cómo está ofuscado el código), y no de una base matemática robusta. Una vez que el método se descubre o se hace ingeniería inversa, la protección desaparece por completo, a diferencia de la criptografía bien diseñada, que sigue siendo segura incluso si el algoritmo es público (principio de Kerckhoff).

V. PUNTO 5: FIRMA DIGITAL EN PYTHON

1.2 Generación del par de claves RSA: Generamos el par de llaves usando RSA con un tamaño de 2048 bits (considerado seguro actualmente) y el exponente público estándar 65537. La llave pública se deriva matemáticamente de la privada, por lo que no es necesario generarla por separado.

1.3 Firma del mensaje con la llave privada: La función de firma toma el mensaje y la llave privada, y usa el método `.sign()` con padding PSS (el esquema recomendado para firmas RSA) y SHA-256 como algoritmo de hash. Internamente, no se firma el mensaje completo directamente, sino su hash: primero se calcula el SHA-256 del mensaje, y ese hash es lo que se firma con la llave privada, produciendo una firma en bytes.

1.4 Verificación de la firma con la llave pública: La función de verificación usa `.verify()` con la llave pública, la firma y el mensaje. Este método no retorna `True/False` directamente: si la firma es válida no hace nada, y si es inválida lanza una excepción. Por eso encapsulamos la verificación en un bloque `try/except`, retornando `True` si no hubo excepción y `False` si se capturó una. Probamos el flujo firmando un mensaje original y verificando que la firma es válida contra ese mismo mensaje.

1.5 Modificación del mensaje y falla de la firma: Tomamos la misma firma generada sobre el mensaje original y la verificamos contra un mensaje alterado (cambiando el nombre del firmante). La verificación falla correctamente, demostrando que la firma está ligada de forma única al contenido exacto del mensaje: cualquier cambio, por mínimo que sea, invalida la firma.

2. Autenticidad y no repudio mediante firma digital: La autenticidad se garantiza porque solo el poseedor de la llave privada puede generar una firma válida; si la verificación con la llave pública correspondiente es exitosa, se confirma que el mensaje fue firmado por esa persona específica. El no repudio se da porque, al ser la llave privada única y (idealmente) secreta, el firmante no puede negar haber firmado el mensaje: cualquiera con la llave pública puede verificar de forma independiente que la firma corresponde a esa llave privada, sin necesidad de confiar en un tercero. Además, como la firma depende del hash del contenido exacto, cualquier intento de alterar el mensaje después de firmado invalida la firma, protegiendo también la integridad del documento.

VI. PUNTO 6: INTRODUCCIÓN PRÁCTICA A BLOCKCHAIN (SIMULACIÓN BÁSICA)

1.1 Estructura de bloque: Definimos una clase `Bloque` que almacena cinco campos: el índice (posición del bloque en la cadena), el timestamp (fecha y hora exacta de creación, tomado automáticamente al instanciar el bloque), los datos (la información a guardar, en este caso mensajes de transacciones ficticias), el hash del bloque anterior (necesario para encadenar la información, con valor "0" para el primer bloque) y el hash propio, calculado automáticamente dentro del constructor al momento de crear el bloque.

1.2 Creación de la cadena con varios bloques: Implementamos una clase `Blockchain` que administra la lista de bloques. Al inicializarse, la cadena se crea automáticamente con un bloque génesis (el primer bloque, sin bloque anterior real). Luego, mediante la función `agregar_bloque`, añadimos dos bloques con información ficticia de transacciones ("Alice envía 10 a Bob" y "Bob envía 5 a Charlie"), donde cada bloque

nuevo toma automáticamente el hash del último bloque de la cadena como su propio `hash_anterior`.

1.3 Cálculo del hash y encadenamiento: El hash de cada bloque se calcula concatenando su índice, timestamp, datos y el hash del bloque anterior, y aplicando SHA-256 sobre ese contenido combinado. Este diseño es lo que genera el encadenamiento: como el hash de cada bloque depende del hash del bloque anterior, cualquier bloque queda matemáticamente ligado a toda la historia previa de la cadena. Al imprimir los bloques de nuestra cadena de prueba, se puede verificar que el `hash_anterior` de cada bloque coincide exactamente con el `hash_propio` del bloque que lo precede.

1.4 Modificación de un bloque y ruptura de la cadena: Para comprobar la inmutabilidad, modificamos directamente el campo `datos` del segundo bloque sin recalcularlo su hash (simulando una manipulación real, donde un atacante alteraría el contenido pero no podría regenerar la cadena de hashes válida sin ser detectado). Al validar la cadena antes de la alteración, el resultado es válido (`True`); después de modificar el dato, la validación detecta la inconsistencia y retorna `False`, ya que el hash almacenado en el bloque ya no corresponde a su contenido actual.

2. Validación de integridad de la cadena (desafío adicional): Implementamos la función `es_valida()`, que recorre toda la cadena verificando dos condiciones en cada bloque: primero, que su `hash_propio` siga correspondiendo al hash recalculado a partir de su contenido actual (detectando alteraciones directas); segundo, que su `hash_anterior` coincida con el `hash_propio` real del bloque previo (detectando rupturas en el encadenamiento). Si cualquiera de las dos condiciones falla en algún bloque, la función retorna `False` de inmediato.

Reflexión: ¿Por qué la inmutabilidad y el consenso son claves en la seguridad de blockchain? La inmutabilidad es clave porque, al estar cada bloque encadenado criptográficamente al anterior mediante su hash, modificar cualquier dato histórico invalida automáticamente el hash de ese bloque y rompe la cadena hacia adelante, haciendo la alteración detectable de inmediato (como se demostró en la prueba). Esto garantiza que el historial de transacciones sea confiable y no pueda reescribirse silenciosamente. El consenso, por su parte, es fundamental en sistemas blockchain reales (distribuidos entre múltiples participantes) porque, aunque un atacante lograra recalcularlo una cadena alterada completa en su propia copia, necesitaría además convencer a la mayoría de la red de aceptar esa versión falsa como válida; sin un mecanismo de consenso, no habría forma de determinar cuál copia de la cadena es la legítima entre múltiples nodos, dejando el sistema vulnerable a versiones conflictivas o fraudulentas.

VII. PUNTO 7: LABORATORIO SOBRE VPN Y HTTPS

Objetivo: Identificar cómo se implementa el cifrado de canal seguro y sus beneficios.

VPN (demostración guiada, no necesariamente desarrollo)

- 1) Usa aplicaciones como ProtonVPN u OpenVPN (puede ser con cuenta limitada).
- 2) Conéctate a una VPN y accede a una página de testeo como <https://ipleak.net/> antes y después.
- 3) Analiza brevemente qué cambia y qué protege una VPN.

HTTPS (ejercicio técnico)

- 1) Utiliza Python (librería requests) para hacer una petición a un sitio web que tenga HTTPS y otro sin HTTPS.
- 2) Revisa la diferencia, los encabezados y errores posibles.
- 3) Verifica cómo se comprueba la validez de un certificado SSL/TLS.

1. Uso de aplicación “ProtonVPN”

Antes de conectar a la VPN:

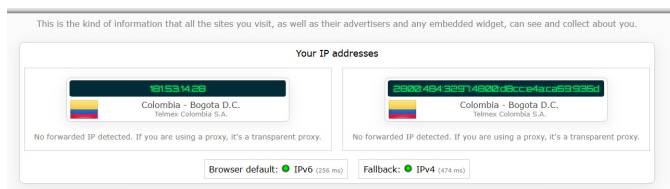


Fig. 1. Direcciones IP antes de conectar a la VPN.

Después de conectar al VPN:

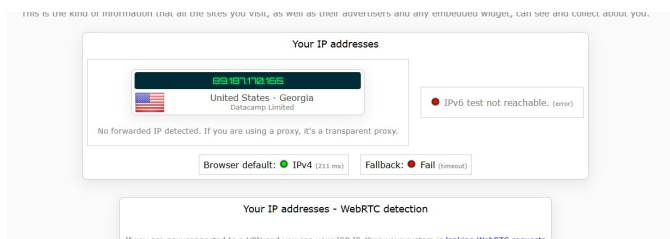


Fig. 2. Direcciones IP después de conectar a la VPN.

Cambios observados

- Se obtuvo un cambio de IP a una pública, por lo tanto pasó de mi IP real a 89.187.170.165.
- Se observa también un cambio geográfico, ahora aparece como Estados Unidos - Georgia, en vez de la ubicación real.
- Se tiene además un cambio del operador real a “Datacamp Limited” que es un proveedor de hosting/datacenter, no un ISP residencial normal. Eso confirma que la IP pertenece a un servidor de la VPN.
- IPv6: pasó de estar disponible a marcar “IPv6 test not reachable (error)”, esto puede ser porque la VPN bloqueó la IPv6 real para evitar que se filtrara.

Qué protege una VPN

- Oculta la ubicación e IP real, cualquier sitio que se visite mientras se esté conectado ve la IP y ubicación del servidor VPN (Georgia, EE. UU.), no la ubicación real.
- Evita filtraciones de IPv6: Al conectarse a la VPN, el IPv6 se bloquea en lugar de dejarse pasar sin cifrar, evitando así que la identidad real del usuario se filtre por ese canal paralelo.
- Cifra el tráfico entre el dispositivo y el servidor VPN: Esta es una de las funciones centrales de una VPN. Al estar conectado a una red pública, como el wifi de un aeropuerto, cualquier persona conectada a esa misma red podría intentar interceptar el tráfico de otros dispositivos y leer datos sin cifrar. Al usar una VPN, todo el tráfico sale del dispositivo ya cifrado antes de pasar por esa red, por lo que aunque alguien logre interceptarlo, solo vería datos ilegibles y no las páginas visitadas ni la información enviada.
- Hay que tener en cuenta que esta protección mencionada anteriormente es distinta al cambio de IP pública: mientras el cambio de IP oculta la identidad ante los sitios web visitados, el cifrado protege el contenido del tráfico ante quienes comparten la misma red local.

VPN. HTTPS (ejercicio técnico)

1. Diferencia entre el sitio con HTTPS y sin HTTPS:

Con HTTPS el servidor presenta un certificado verificado por una autoridad certificadora, y todo lo que viaja entre el cliente y el servidor va cifrado, así que si alguien intercepta el tráfico no puede leer nada. Con HTTP no hay certificado ni cifrado, los datos viajan en texto plano y cualquiera que esté en la red podría verlos.

2. Encabezados y errores posibles:

En la respuesta HTTPS aparece el encabezado Strict-Transport-Security, que le dice al navegador que de ahí en adelante siempre use HTTPS con ese sitio y nunca HTTP. En la conexión HTTP este encabezado no aparece, porque no tiene sentido ahí. En cuanto a errores, al intentar conectarnos por HTTP dentro de Colab la petición falló con un error de red, esto muestra cómo muchas infraestructuras hoy en día ya casi no dejan pasar tráfico sin cifrar, porque HTTPS es el estándar. También forzamos un error de certificado usando expired.badssl.com (un sitio de pruebas con certificado vencido), y efectivamente lanzó un SSLError. Con esto se confirma que requests, igual que los navegadores, rechaza automáticamente certificados inválidos en vez de dejar pasar la conexión de forma insegura.

3. Cómo se comprueba la validez de un certificado

SSL/TLS: Conectándonos directamente al puerto 443 del servidor y pidiendo su certificado, podemos ver a quién fue emitido, qué autoridad certificadora lo emitió, y sus fechas de validez. La verificación revisa automáticamente que el certificado no esté vencido, que corresponda al dominio al que nos conectamos, y que venga de una autoridad confiable. Si algo de eso falla, se genera un error de seguridad (como el SSLError que provocamos) en vez de permitir la conexión.

VIII. PUNTO 8: INVESTIGACIÓN: ESTADO DEL ARTE SOBRE VPN, HTTPS, BLOCKCHAIN Y FIRMA DIGITAL

1. Elegimos la tecnología de Blockchain

3. El artículo seleccionado es:

Contenido Multimedia con tecnologías emergentes como el blockchain para la creación de NFT, estudio de caso: Compañero Virtual.ETB S.A E.S.P

4. Lean el documento seleccionado y respondan estas preguntas para construir un “estado del arte”:

a. **¿Cuál es la aplicación concreta presentada?** Este proyecto propone el análisis del estado de las tecnologías emergentes Blockchain y NFT en Colombia, desde la perspectiva de la Ingeniería en Multimedia y el desarrollo de un estudio de caso, con un compañero virtual como prototipo de asistente/acompañante interactivo, diseñado como contenido multimedia ejecutable y descargable desde la red que permita mostrar los hallazgos y proponer un modelo de negocio para gestionar de forma sostenible contenido multimedia con Blockchain y NFT.

b. **¿Qué problema o necesidad aborda?** Describen el problema de la monetización limitada del contenido multimedia que se suma a la desconfianza del público provocada por el uso especulativo de los NFTs, la manipulación de precios por ciertos grupos y el temor a la inversión debido a la proliferación de obras generadas por inteligencia artificial. Por ende se genera la necesidad de promover la investigación y educación sobre el verdadero potencial de la tecnología blockchain y los NFTs, así como fomentar la creación y adquisición de contenidos multimedia reales para lograr un crecimiento sostenible y equitativo del sector.

c. **¿Qué desafíos o dificultades se mencionan?**

- **Incompatibilidad en Unity:** Se descartó el uso de la librería Newtonsoft (desarrollada por ChainSafe) debido a que no se contaba con la forma de validar la conectividad directa con Unity ni de certificar el cumplimiento de las condiciones del contrato.
- **Problema con el tamaño de los tokens:** Al intentar manipular el NFT generado a través de OpenSea dentro de Unity, el motor no pudo soportar el tamaño de bytes tan elevado del ID del token (un número de 78 dígitos).
- **Centralización de datos (Riesgo de pérdida de información):** Varios archivos y metadatos alojados en servidores privados corren el riesgo de perderse o quedar inaccesibles si los servidores eliminan los datos, lo que requirió considerar soluciones descentralizadas como IPFS.

d. **¿Cuáles son las principales conclusiones o recomendaciones de la publicación?**

- **Viabilidad y rentabilidad en Colombia:** Existe un panorama favorable para la creación y gestión sostenible de contenidos multimedia con Blockchain y NFT, respaldado por la amplia documentación disponible, empresas facilitadoras y herramientas que mitigan los costos de creación.

- **Seguridad y verificación sin intermediarios:** La tecnología ofrece una alta seguridad frente a hackeos o copias no autorizadas, permitiendo verificar la autenticidad y emisión de los productos multimedia sin depender de un servidor centralizado.

- **Necesidad de valor agregado:** Un NFT por sí solo no garantiza valor en el mercado, para ser comercializable y rentable, debe incluir utilidades reales como beneficios, acceso a eventos, descuentos o soporte para su reproducción en aplicaciones específicas.

- **Eficiencia en costos según el enfoque:** Crear NFTs apoyándose en marketplaces existentes resulta económico o tal vez gratuito, mientras que programar y desplegar un contrato propio sin optimizar puede elevar drásticamente las tarifas de gas.

e. **¿Qué ideas pueden ser útiles para la universidad u organizaciones similares?**

- **Sedes y laboratorios Web3 / Sandbox universitario:** Implementar entornos de prueba dentro de la universidad para que los estudiantes de Ingeniería en Multimedia experimenten con smart contracts.
- **Creación de micro credenciales y certificados en Blockchain:** Utilizar contratos inteligentes para emitir certificados académicos, insignias o diplomas como NFTs no transferibles.

5. Elaboren un resumen breve (media página) con los resultados de su análisis. Incluyan:

Nombre del Trabajo de grado: Contenido Multimedia con tecnologías emergentes como el blockchain para la creación de NFT

Fecha: 2023-05-04

Autores: Pinzón Granados, Jose Arturo; Gutierrez Hermida, Diego Alejandro

[1] J. A. Pinzón & D. A. Gutierrez Contenido Multimedia con tecnologías emergentes como el blockchain para la creación de NFT, estudio de caso: Compañero Virtual.ETB S.A E.S.P. [online]. Disponible en: <https://hdl.handle.net/10654/45049>

Finalmente el trabajo analiza la rentabilidad y sostenibilidad de los contenidos multimedia en Colombia mediante el uso de tecnologías Blockchain y NFT. A través del desarrollo de un caso de estudio centrado en un acompañante virtual interactivo (Shimeji), los autores exploraron la integración técnica de modelos 2D vectorizados, animación y motores como Unity con la red de bloques. Entre los hallazgos principales destacan:

- Colombia presenta un entorno favorable para monetizar proyectos digitales con Blockchain, pero los NFTs requieren de un valor agregado real (como utilidades dentro de apps o acceso a beneficios) para mantener su valor comercial y evitar caer en la mera especulación.
- El costo de implementación de un contrato inteligente propio mal optimizado genera un consumo elevado de tarifas de gas en comparación con el uso de un contrato optimizado en un marketplace como OpenSea.
- Para evitar la pérdida o centralización de metadatos alojados en servidores privados, la adopción de tecnologías

descentralizadas como IPFS resulta indispensable para la preservación de la información de los NFTs.

“Se concluye que la Ingeniería en Multimedia tiene un papel importante que desempeñar en la promoción del uso de tecnologías emergentes como la Blockchain y los NFTs en Colombia. Por eso es necesario fomentar la investigación y el desarrollo en las áreas de la innovación tecnológica desde El matices o El Matices, se pueden crear oportunidades para el crecimiento económico en el país.”

REFERENCES

- [1] J. A. Pinzón and D. A. Gutierrez, “Contenido Multimedia con tecnologías emergentes como el blockchain para la creación de NFT, estudio de caso: Compañero Virtual.ETB S.A E.S.P.,” [Online]. Available: <https://hdl.handle.net/10654/45049>
- [2] Kaspersky, “¿Qué es una VPN y cómo funciona?,” Centro de Recursos Kaspersky, 2026. [En línea]. Disponible: <https://latam.kaspersky.com/resource-center/definitions/what-is-a-vpn>.
- [3] Mozilla, “An overview of HTTPS,” MDN Web Docs, 2026. [En línea]. Disponible: https://developer.mozilla.org/en-US/docs/Web/Security/Transport_Layer_Security.
- [4] DigiCert, “¿Qué es un certificado SSL?,” 2026. [En línea]. Disponible: <https://www.digicert.com/es/what-is-an-ssl-certificate>.
- [5] Chromium Project, “badssl.com,” 2026. [En línea]. Disponible: <https://badssl.com/>. [Accedido: 20-ago-2026].
- [6] Python Software Foundation, “ssl — Envoltorio TLS/SSL para objetos socket,” Documentación de Python, 2026. [En línea]. Disponible: <https://docs.python.org/es/3/library/ssl.html>. [Accedido: 20-ago-2026].